

FreshMart – The Data Challenge

Current Scenario

FreshMart wants to bring data from its physical stores, online shopping platform, and customer loyalty program into one central data platform. These sources have different data arrival patterns and freshness requirements. The company also faces late, incomplete, duplicate, corrected, and differently structured data. FreshMart therefore needs a reliable ingestion approach that handles data changes, validation, schema changes, and avoids unnecessary reprocessing.

Problems:

1. Different Data Arrival Frequencies

- Store data arrives at the end of the day, online data arrives continuously, and loyalty data includes new or updated customer information.

2. Different Freshness Requirements

- Different teams need data at different speeds, from end-of-day store information to near-real-time online activity.

3. Late-Arriving Store Data

- Some store information arrives later than the expected delivery time.

4. Incomplete Records

- Some incoming records are missing required information or fields.

5. Duplicate Transactions

- Some transactions appear more than once in the incoming data.

6. Corrections to Existing Data

- Stores can correct information that was already shared and ingested earlier.

7. Loyalty Program Data Changes

- Some loyalty information is recorded differently after a recent program change, creating schema compatibility concerns.

8. Unnecessary Reprocessing

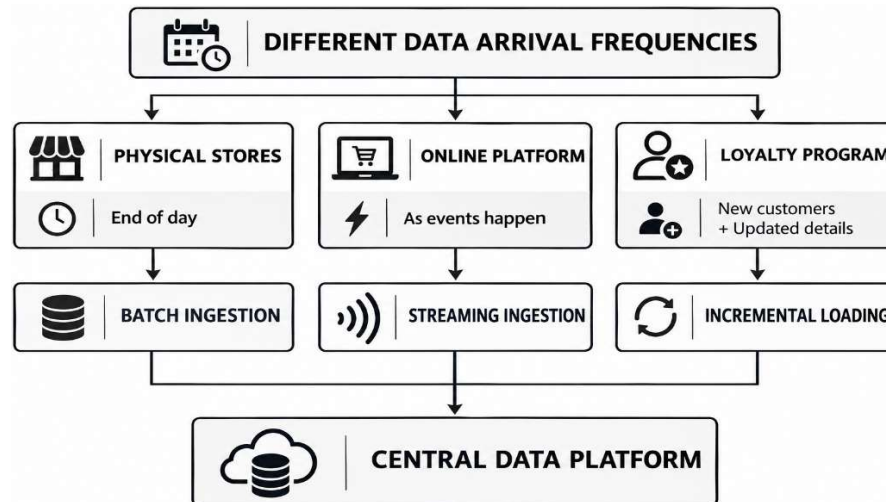
- FreshMart wants to avoid repeatedly processing information that has not changed.

Solutions:

Problem 1. Different Data Arrival Frequencies

Solution: Batch, Streaming & Incremental Loading

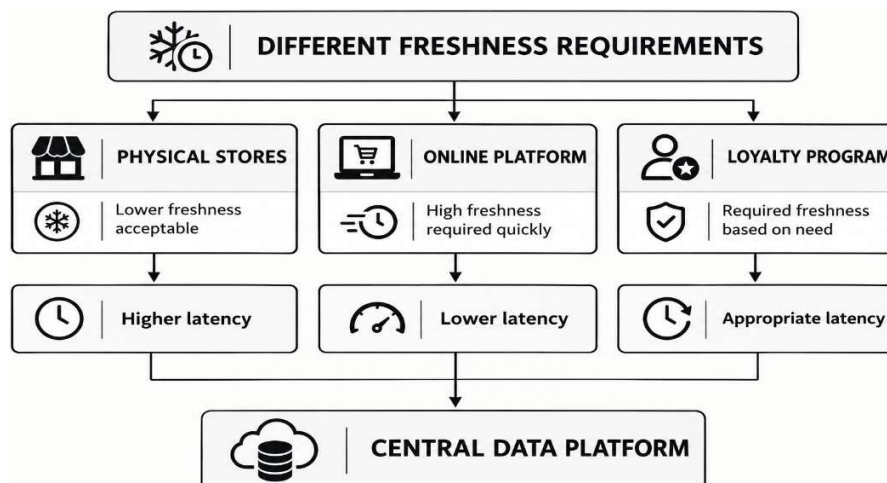
- **Batch Ingestion:** Use for physical store data because it is shared at the end of the day.
- **Streaming Ingestion:** Use for online orders, cancellations, and purchases as they happen.
- **Incremental Loading:** Process new loyalty customers and changed loyalty details without repeatedly processing unchanged data.



Problem 2. Different Freshness Requirements

Solution: Latency & Freshness Management

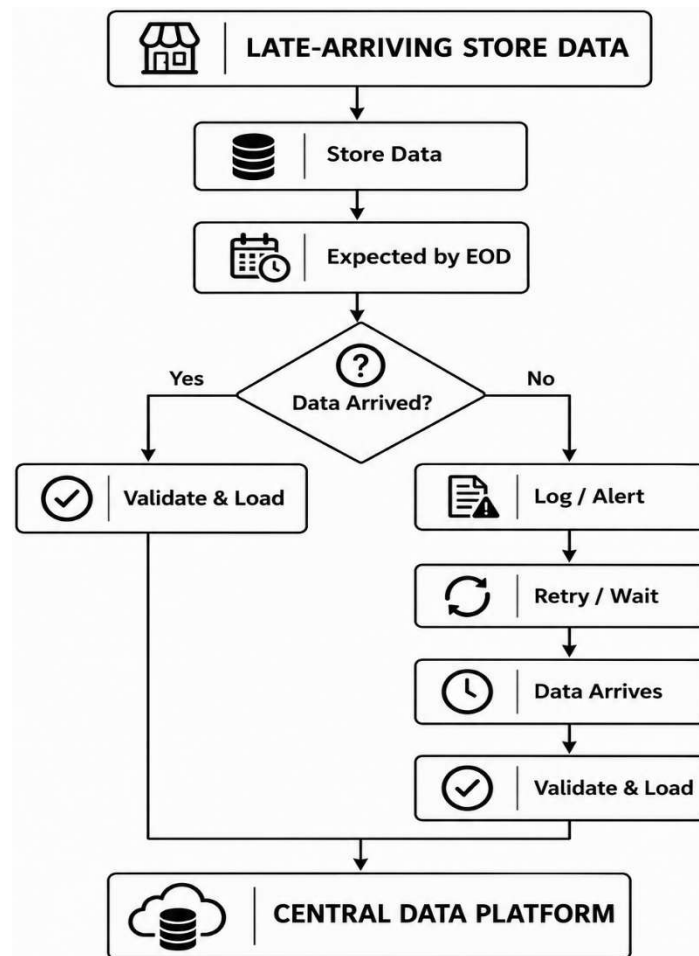
- **Low Latency:** Provide online orders, cancellations, and completed purchases quickly because the online team needs near-real-time visibility.
- **Higher Latency:** Allow store information to arrive later because it is shared at the end of the day.
- **Appropriate Freshness:** Process loyalty updates according to the business's required freshness, without assuming real-time processing when the case does not specify it.



Problem 3. Late-Arriving Store Data

Solution: Ingestion Error Handling

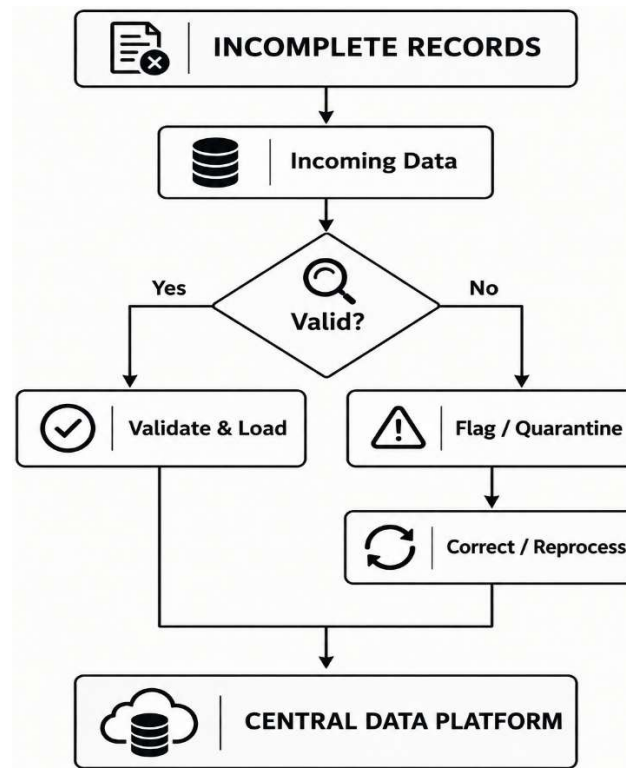
- **Detect & Monitor:** Check whether expected store data has arrived by the expected time and log/alert when it is late or missing.
- **Retry & Reprocess:** Retry or wait for delayed data, then validate and load it when it arrives; reprocess it if the original ingestion was missed.



Problem 4. Incomplete Records

Solution: Data Validation

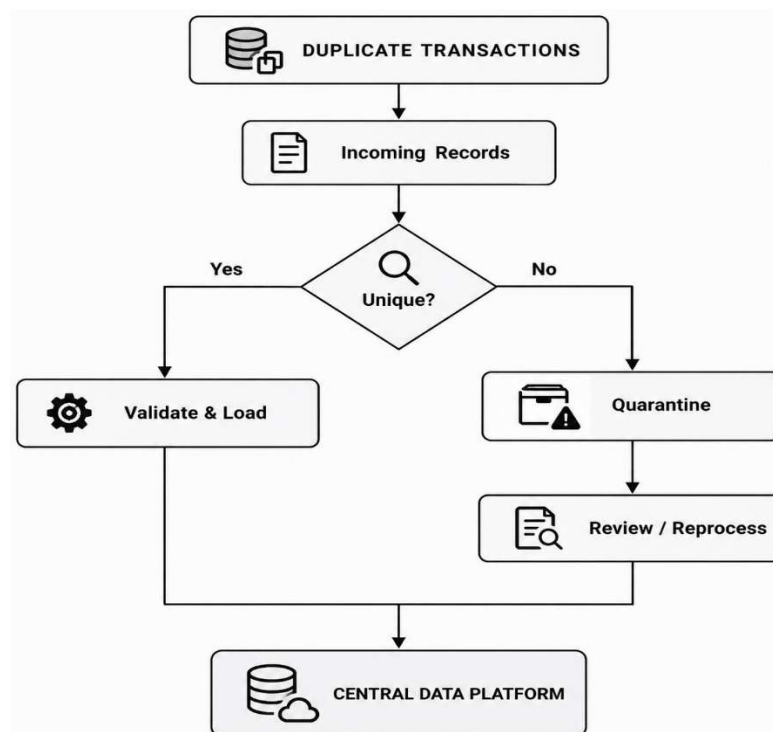
- **Validate Required Fields:** Check incoming records for missing required values and identify incomplete records before loading.
- **Handle Invalid Records:** Flag or quarantine incomplete records for correction and prevent them from contaminating the central data platform.



Problem 5. Duplicate Transactions

Solution: Data Validation & Duplicate Detection

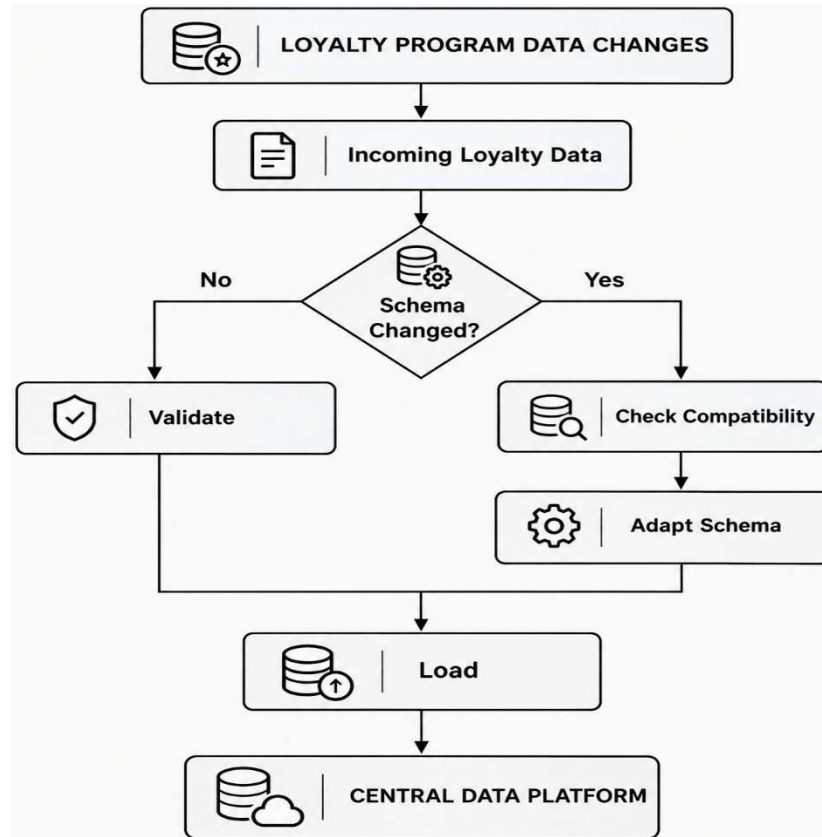
- **Detect Duplicates:** Validate incoming transactions using a unique identifier such as `transaction_id` to identify records that appear more than once.
- **Prevent Duplicate Loading:** Allow only valid unique records into the central platform and quarantine duplicate records for review or reprocessing.



Problem 6. Loyalty Program Data Changes

Solution: Schema Handling & Compatibility

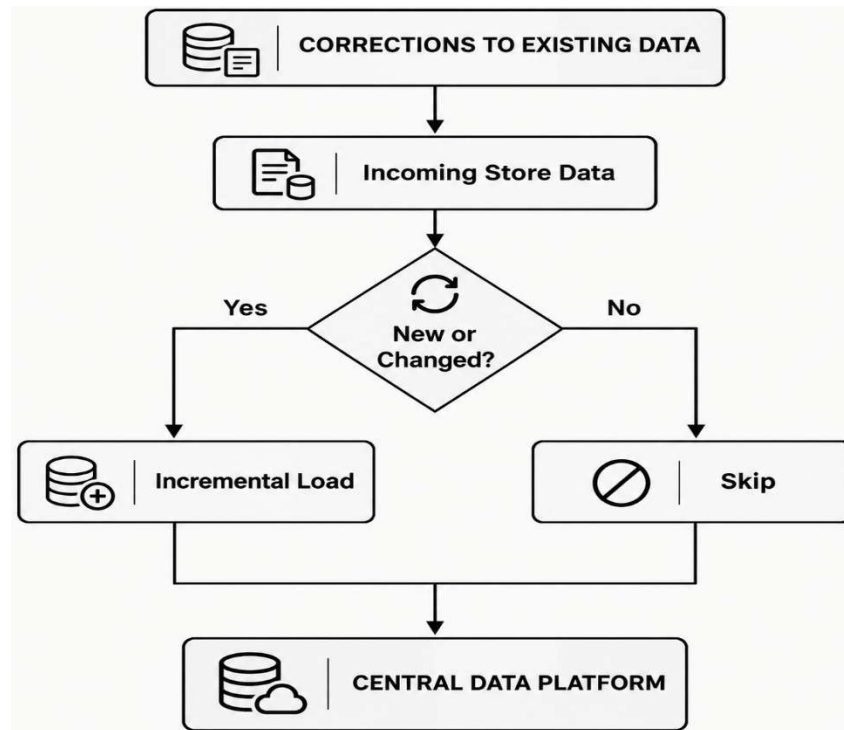
- **Detect Schema Changes:** Compare incoming loyalty data with the expected schema to identify changed fields, formats, or structures.
- **Validate & Adapt:** Check whether the changed data is compatible; adapt the schema or transformation rules when required before loading it into the central platform.



Problem 7. Corrections to Existing Data

Solution: Incremental Loading

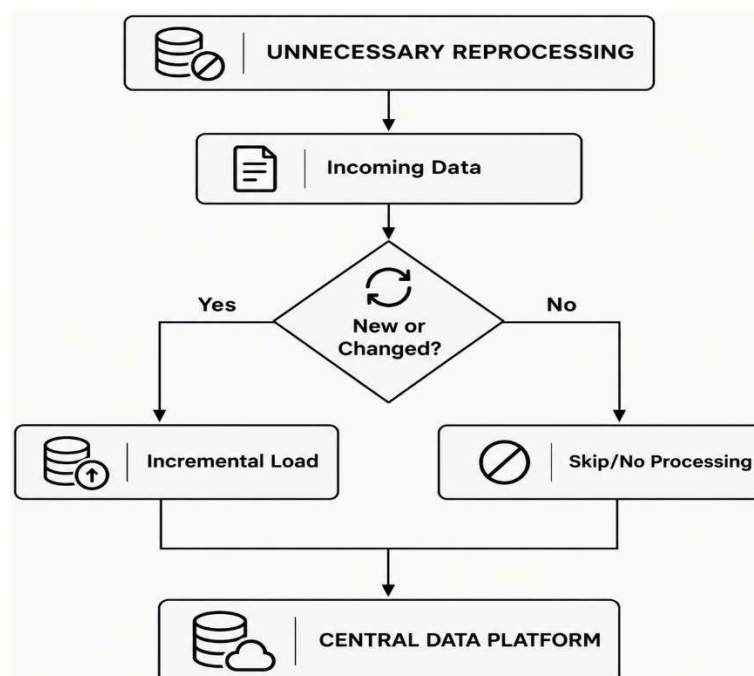
- **Detect Changes:** Identify store records that have been corrected or modified after their original ingestion.
- **Process Only Changes:** Load the new or updated records instead of reprocessing the complete historical dataset.



Problem 8. Unnecessary Reprocessing

Solution: Incremental Loading

- **Identify Changes:** Track new or modified records so only data that has changed is selected for processing.
- **Skip Unchanged Data:** Avoid reprocessing records that have not changed, reducing unnecessary processing and resource usage.



Proposed Architecture:

